

Lab Report 7: Intro to Photonic Quantum Computing with Xanadu

Bryan Fernandez, Pranav Reddy, Nate Barnaby, Gavin Kesler

Electrical and Computer Engineering Department, University of California, Santa Barbara

1 Abstract

In this lab we will dive into the basics of photonic quantum computing using the Strawberry Fields library and Xanadu's cloud quantum photonic simulator. By constructing and simulating quantum circuits with squeezed vacuum states, beamsplitters, and photon number measurements, we will explore statistical patterns of states in the Fock basis. Important topics such as mean photon number, variance, and distributions will be analyzed through single and two mode quantum programs. This lab serves as an introduction to continuous variable quantum computing, enabling us to create quantum states, utilize gates, and visualize outcomes through photon number statistics.

2 Introduction

Quantum computing through photonics opens our eyes to scalable quantum information processing. Unlike the qubit based approaches we have taken previously in the class, photonic systems can track information in variables like amplitude and phase. We can contort these variables using linear optical components and non classical light sources. This lab introduces us to the foundational photonic quantum programming through Strawberry Fields, a python quantum simulation library developed by the company Xanadu. Through our lab, we will construct quantum circuits that will utilize squeezed vacuum states, two-mode squeezing, and beamsplitters to explore quantum interference, entanglement, and photon number generation.

Through analyzing the mean and variance of photon number distributions and the

visualizations our python code grants us through Fock-space probabilities, we will know the effect of quantum operations on photonic states. This lab serves as the basics towards understanding how variable systems can be used to represent and process quantum information.

3 Procedure & Results

We will first start our program and connect to Xanadu cloud.

```
# Connecting to the Xanadu Cloud
import xcc
xcc.Settings(REFRESH_TOKEN=redacted for
security).save()
import xcc.commands
xcc.commands.ping()
```

Image 1: Connecting to Xanadu Cloud

Task 1: Creating a 2-Mode Program

We will begin by creating a two mode quantum program using the Strawberry Fields library. We set up the simulation with two optical paths, called modes. We need to initialize the modes in the vacuum state, a state with zero photons.

We then will apply a two-mode squeezing gate, an operation that creates quantum entanglement between two modes. This will make photon counts correlated. Two mode squeezing is essential for generating entangled states in variable quantum computing.

We will then measure mode 1 using the Fock measurement, which is a type of measurement that will tell us exactly how many photons are detected in that mode. The output will be

probabilities showing the quantum properties of our setup.

```
import scipy.integrate as _int

_int.simps = _int.simpson # make old name
available

import strawberryfields as sf
from strawberryfields import ops

#Task 1
# create a 2-mode quantum program
prog = sf.Program(2)

with prog.context as q:
    ops.S2gate(0.50) | (q[0], q[1])
    ops.MeasureFock() | q[0]
```

Image 2: Creating two mode quantum program

Task 2: Quantum Circuit Diagram

We will then create a quantum circuit diagram to visualize the program from Task 1. This diagram will include two input modes in a vacuum state. A two mode squeezing operation between the two and finally a photon counting measurement on mode 1.

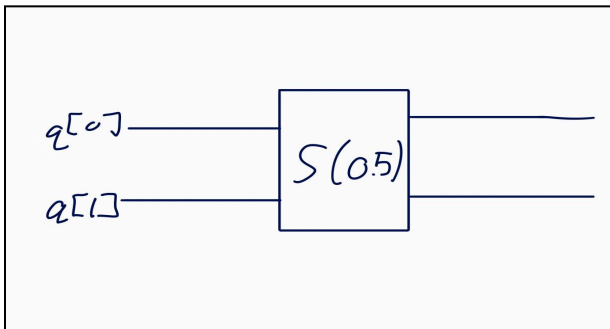


Image 3: 2-mode squeezing circuit diagram

Task 3: Initializing the Fock Backend

To simulate our quantum circuit, we will need to create a Fock backend using a 5 cutoff. The cutoff defines the highest number of photons the simulator can have for every mode. This allows us to see the quantum state as a Hilbert space.

```
# Task 3
eng = sf.Engine("fock",
backend_options={"cutoff_dim": 5})
result = eng.run(prog)
state = result.state
```

Image 4: Program to initialize the Fock Backend

Task 4: Size of Density Matrix

We call on a function to inspect the size of the density matrix for the two mode system we programmed in task 2.

```
# Task 4
state.dm().shape
```

Image 5: Determining the shape of the density matrix

The function outputs a result of:
(5, 5, 5, 5)

Task 5: Photon Statistics for 3 Mode

For this task we will create a quantum program with three photonic modes. Each mode will be created with a single mode squeezing gate with a parameter of $r = 0.54$. In order to generate interference between the modes, two beamsplitter gates should be applied to couple modes (0 and 2) and (1 and 2). We will simulate the outcomes of the circuit using the Fock backend with a cutoff dimension of 10.

```
# Task 5
prog = sf.Program(3)

with prog.context as q:
    ops.Sgate(0.54) | q[0] # ops.Sgate(0.54,
np.pi/2) | q[0]
    ops.Sgate(0.54) | q[1]
    ops.Sgate(0.54) | q[2]
    ops.BSgate(0.43, 0.1) | (q[0], q[2])
    ops.BSgate(0.43, 0.1) | (q[1], q[2])

eng = sf.Engine("fock",
backend_options={"cutoff_dim": 10})
```

```
result = eng.run(prog)
state = result.state
```

Image 6: Program for 3 mode quantum circuit

After simulating the 3 mode circuit described in task 5 in the procedure, we computed the mean and variance of the photon number in mode 0. The results were the following:

- Mean photon number: 0.3175
- Variance: 0.8167

These numbers show us the quantum state produced by the circuit. Because the variance is higher than the mean, this indicates super-Poissonian Statistics, which is our expected result. This demonstrates the non-classical states of light. This behavior shows that the system is indeed exhibiting quantum squeezing, where number count fluctuations are reduced below in one quadrature.

Because we used a three mode squeezing gate circuit and coupled the modes with beamsplitters, the quantum state holds photon number correlations. The smaller mean and variance counts indicate a low overall photon count in the mode because of the squeezing. These numbers show that the simulation created a non-classical state and proves the behavior expected from squeezed vacuums.

Task 6: Photon-Number Statistics

To see how different states affect the photon number distributions, we will examine three states: a squeezed vacuum state, a coherent state, and a Fock state. Below, we will analyze and mean and variance of each distribution and explain why for the given state.

1. Squeezed Vacuum State distribution

The squeezed vacuum state was generated using `Sgate(r)` in the program and visualized through `generate_fock_chart()`. This state seems to pick even photon numbers because of parametric down conversion, resulting in a distribution where odd number states have close to 0 probability. The distribution shows a peak near $n=0$ and falls quickly. The photon number value is relatively low at 0.318. Since the state is centered around 0 and only even photon numbers are significant, this shows us that pair creation is a characteristic of squeezing. This distribution was non classical and the mean/variance are low. The state is not a photon eigenstate.

```
from strawberryfields.plot import
generate_fock_chart
from strawberryfields.ops import Fock,
Coherent, Sgate
import plotly.io as pio
import numpy as np

cutoff = 10

fig = generate_fock_chart(
    state,
    modes=[0],
    cutoff=cutoff
)
pio.show(fig)
```

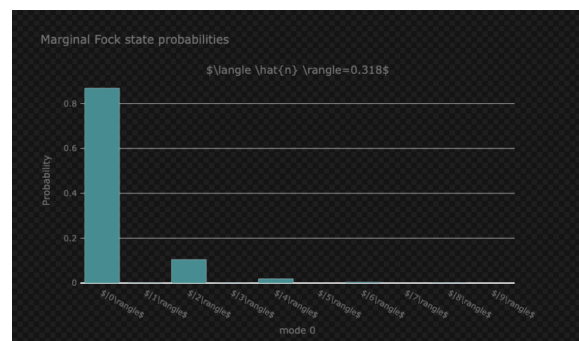


Image 7: Squeezed Vacuum State Code and Graph

2. Coherent State Distribution

The coherent state was programmed using `Coherent(alpha, 0)`, with α being 2. This is supposed to simulate a classical laser field. The probability distribution was accessible through Matplotlib functionality.

We observed that the distribution peaks around $n=4$. This also matches the mean photon number of 4. The variance also is equal to 4. For coherent states, the photon distribution follows a Poisson distribution where the variance is the same as the mean.

This represents a classical state. As α were to increase, both the mean and variance will increase proportionally. This behavior is probably indicative of coherent state nature.

```
import matplotlib.pyplot as plt
prog = sf.Program(1)
n_photon = 8 # photon number state
alpha = 2.0
r = 0.5
phi = 0.0

with prog.context as q:
    Coherent(alpha, 0) | q[0]

# 2) Run on the Fock backend
cutoff = 20
eng = sf.Engine("fock",
    backend_options={"cutoff_dim": cutoff})
result = eng.run(prog)
state = result.state

# 3) Build the P(n) array via a short
loop
probs = np.array([ state.fock_prob([n])
    for n in range(cutoff) ])
```

```
...
# 5) Add mean and variance calculation
mean_n = np.sum(n * probs)
var_n = np.sum((n - mean_n)**2 * probs)
```

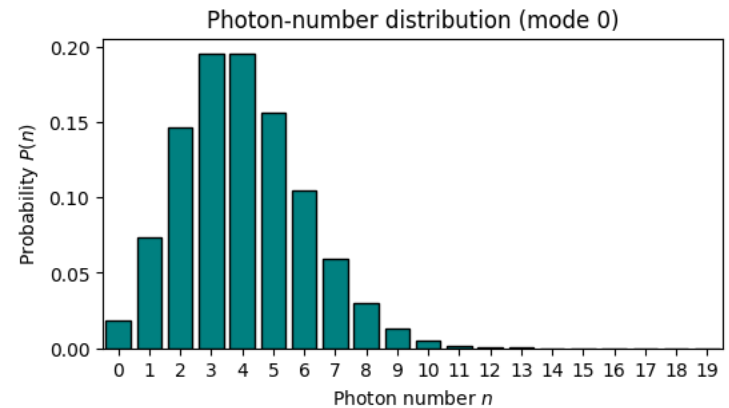


Image 8: Coherent State Code and Graph

3. Fock State Distribution

For the final part, we created a Fock state where $n=8$. This means there are 8 photons with no uncertainty. The resulting photon distribution shows this as the probability of measuring eight photons is one. The mean photon number was calculated to be $n=8$. The variance was 0. This is consistent with the definition of a Fock state as the state number has no spread in its distribution. The coherent and squeezed states are different because their probabilities are spread out. The Fock state is sharply peaked and represents the classical, deterministic quantum state for photon counts. This result shows the baseline in quantum states with uncertainties.

```
# Photon-number statistics demo
import sys
!{sys.executable} -m pip install
matplotlib

import numpy as np
import matplotlib.pyplot as plt
import strawberryfields as sf
```

```

from strawberryfields.ops import Fock,
Coherent, Sgate
n_photon = 8
alpha = 2.0
r, phi = 0.5, 0.0
cutoff = 20

prog = sf.Program(1)
with prog.context as q:
    Fock(n_photon) | q[0]
eng = sf.Engine("fock",
backend_options={"cutoff_dim": cutoff})
result = eng.run(prog)
state = result.state
probs = np.array([state.fock_prob([n])
for n in range(cutoff)])
n = np.arange(cutoff)

mean_n = np.sum(n * probs)
var_n = np.sum((n - mean_n)**2 * probs)

print(f"Mean photon number <n> :
{mean_n:.3f}")
print(f"Variance Var(n) :
{var_n:.3f}")

```

variance. Our results confirmed that Fock states have fixed photon statistics. Coherent states align with Poissonian statistics with equal mean and variance. Finally, squeezed vacuum states show quantum correlations with even photon distributions. These observations show the distinct nature of quantum light and give us a foundation to understand the non classical states used in optics.

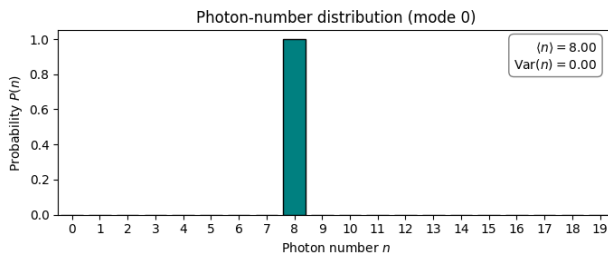


Image 9: Fock State Distribution Graph and Code

5 Conclusion

In this lab, we explored the photon statistics of three quantum states. The Fock, coherent, and squeezed vacuum states. Using the Strawberry Fields library we were able to program and visualize each state, calculate their photon distributions along with their mean and